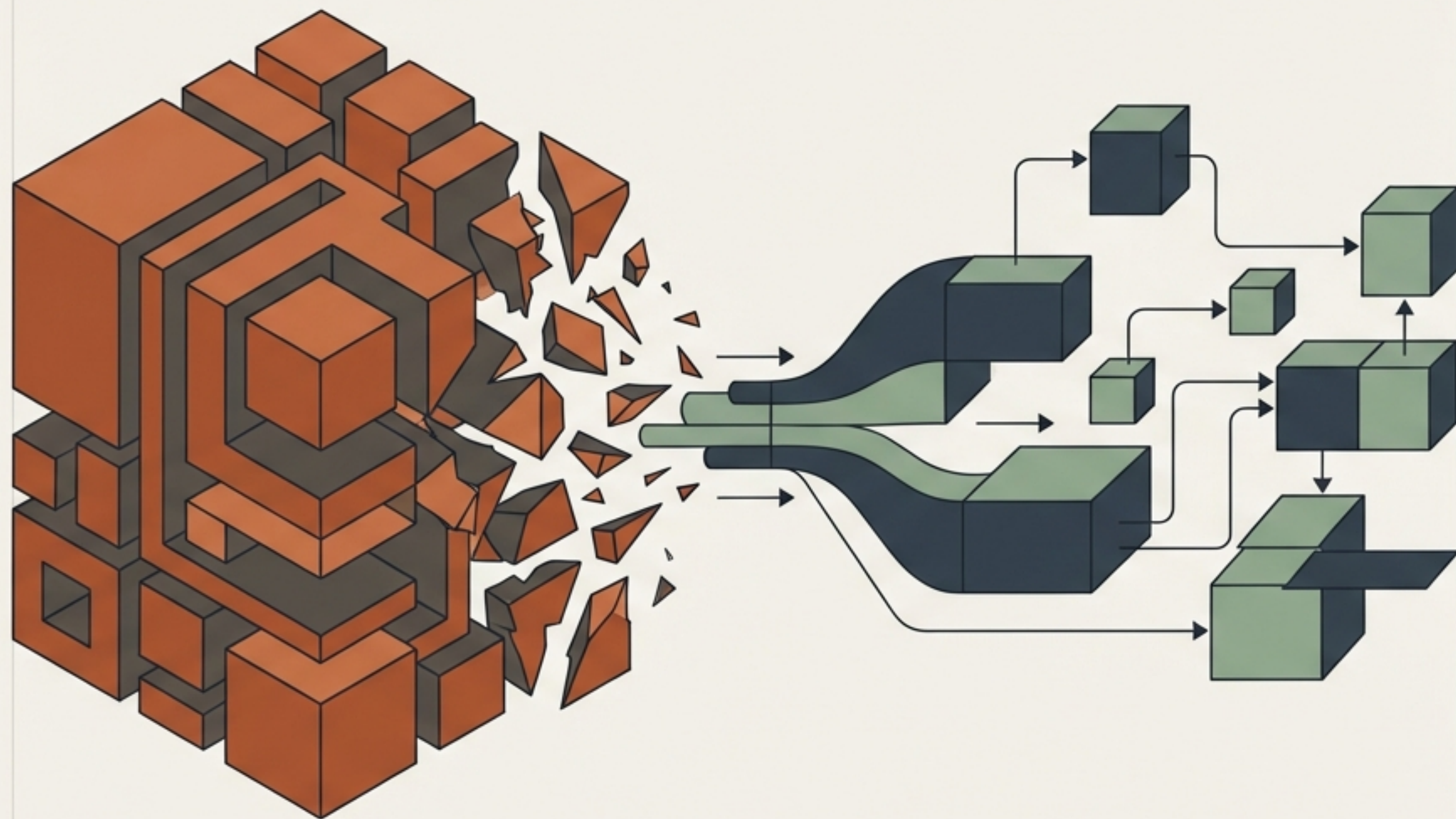


Escape the YAML Trap

Stop writing configuration
files and start writing
infrastructure programs.



DevOps Accidentally Reinvented Lisp

Legacy Configuration (YAML/HCL)

```
resource "aws_instance" "app" {  
  count      = length(var.instances) + 2 ? 0 : 5;  
  ami        = "${local.base_ami}-${var.region}-v3";  
  instance_type = var.env == "prod" ? "t3.large"  
    : lookup(var.types, var.env, "t2.micro");  
  
  tags = {  
    Name      = "${var.project}-app-${count.index}";  
    Environment = var.env;  
    "Owner Email" = "devops@company.com"  
  };  
  provisioner "local-exec" {  
    command = "echo ${self.public_ip} > ip_address.txt"  
  }  
  lifecycle {  
    create_before_destroy = true  
  }  
}  
  
# Complex, error-prone string-based logic and ad-hoc looping
```

CLUNKY STRING LOGIC

COMPLEX LOOPS & CONDITIONS

Structured Data (Clojure)

```
(def app-config  
  :environment :prod  
  :region "us-east-1"  
  :instances  
    {:type "t3.large"  
     :count 5  
     :tags {:name "app-cluster"  
            :owner "devops"}})  
  :ami  
    {:base "ami-xyz123"  
     :version "v3"})  
  
(defn provision-instance [config]  
  (let [[:keys [type count tags]] (:instances config)]  
    (dotimes [i count]  
      (aws/run-instance  
        {:type type  
         :tags (assoc tags :index i)  
         :ami (str (-> config :ami :base) "-"  
                   (-> config :ami :version))})  
      ))))
```

CLEAN DATA STRUCTURE

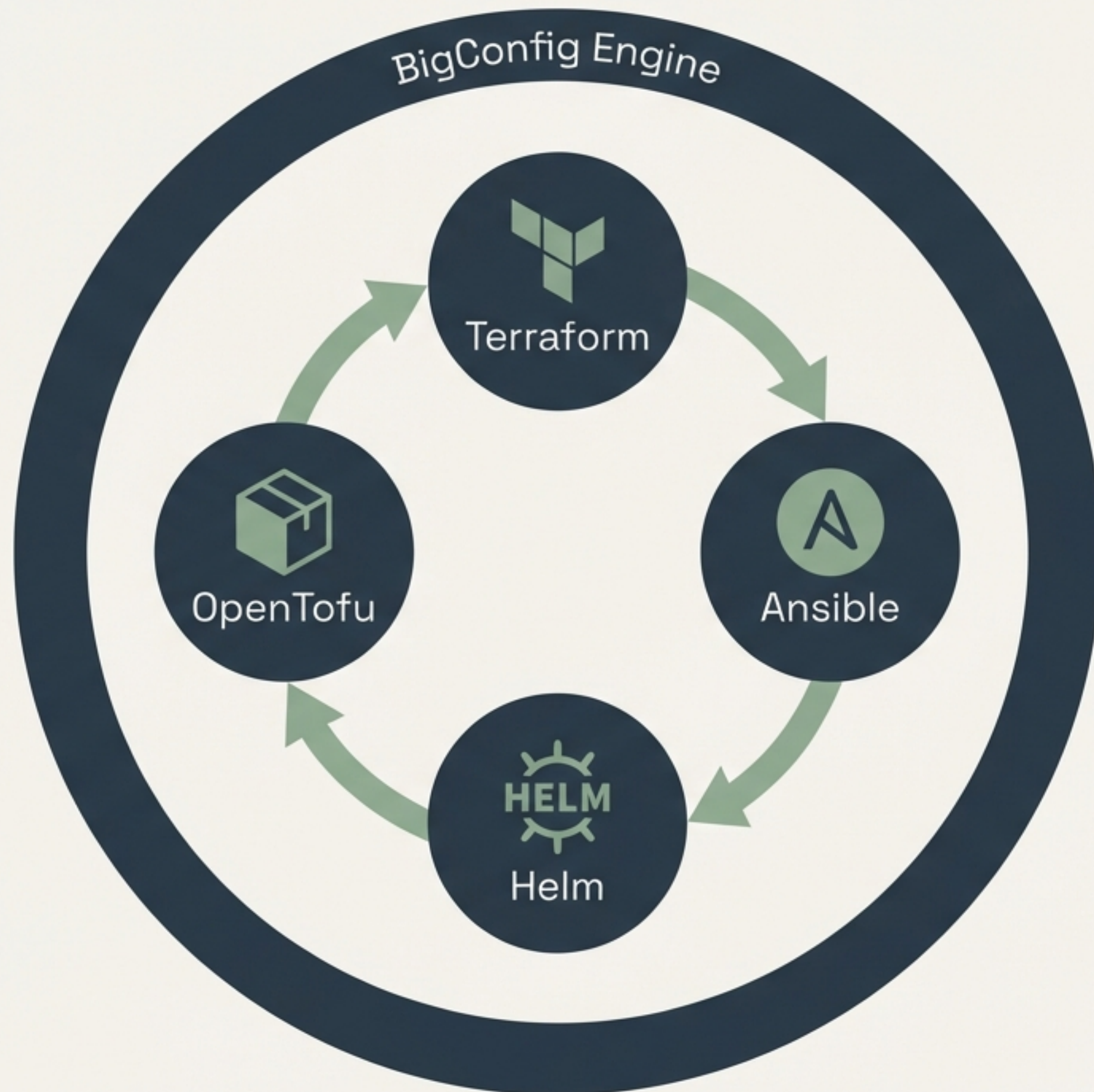
MINIMAL SYNTAX

ELEGANT TRANSFORMATION

Greenspun's Tenth Rule warns that complex systems inevitably reinvent a bug-ridden, slow implementation of half of Common Lisp. By forcing loops and variables into YAML and HCL (Terraform, Ansible, Helm), we trapped ourselves in ad-hoc, un-debuggable configuration languages.

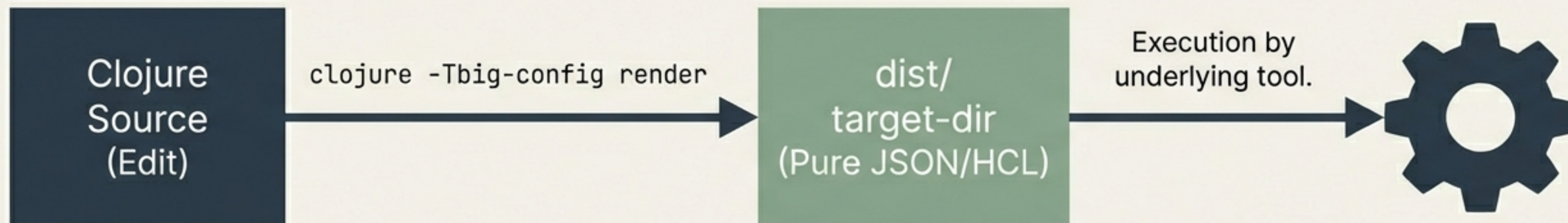
Assimilate, Don't Reinvent

BigConfig doesn't replace your ecosystem. It uses Clojure—a modern, production-grade Lisp—to wrap, generate, and orchestrate existing tools via Babashka, turning fragmented CLI scripts into functional, programmable workflows.



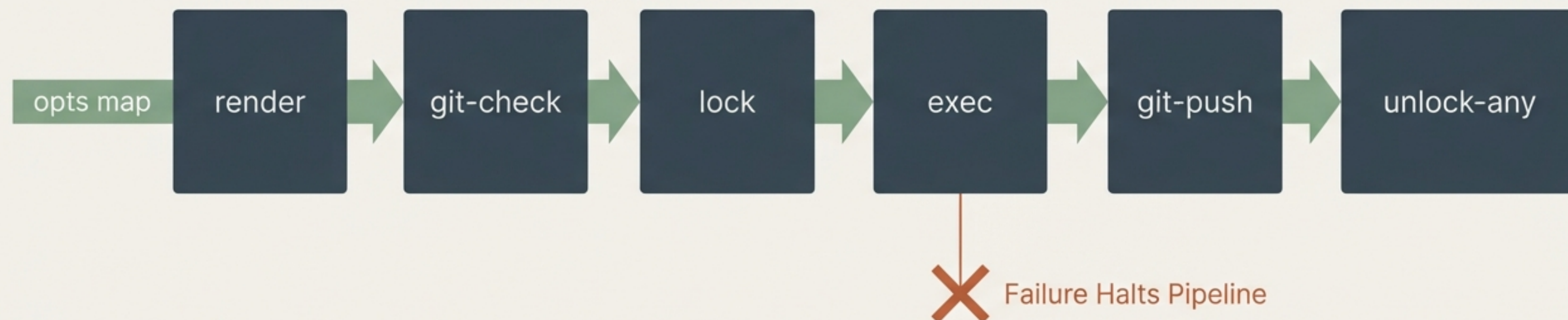
The Zero-Cost Build Step

Just as TypeScript elegantly compiles to JavaScript without breaking the inner dev loop, BigConfig templates generate JSON or HCL dynamically. It's an invisible abstraction layer with zero runtime overhead.



Anatomy of a BigConfig Workflow

Workflows orchestrate infrastructure safely. A unified opts map threads through predefined steps—rendering configs, verifying Git state, acquiring execution locks, executing the tool, and automatically pushing commits.



The Invisible Wrapper

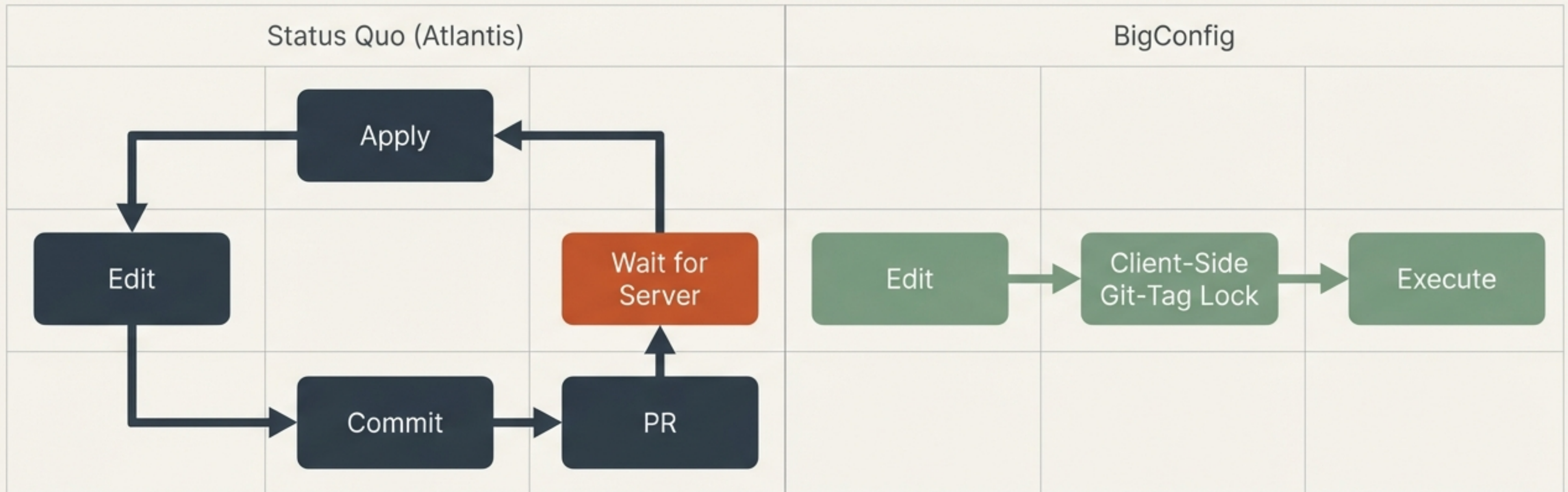
Make the build step vanish. A simple shell alias wraps your standard CLI commands in a safety-first GitOps pipeline, applying changes locally with automated syncing, locking, and execution.

```
alias tofu='bb -- render git-check  
lock exec git-push unlock-any --  
alpha prod tofu'
```



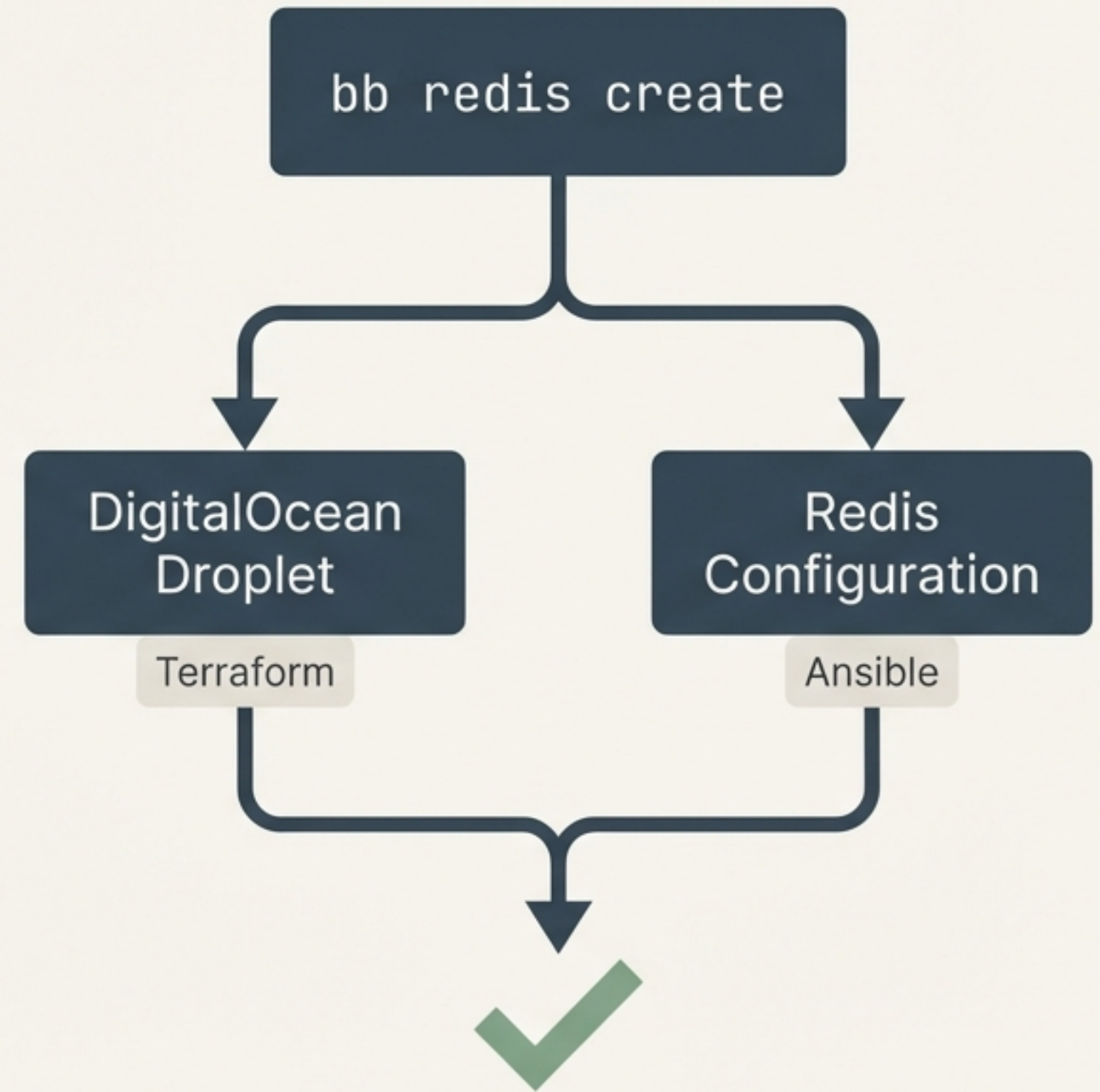
Decentralized Coordination Without the Server

PR-driven workflows break Trunk-Based Development. BigConfig replaces Atlantis by using Git repository tags for exclusive, client-side pessimistic locking (big-config.lock/owner). **Coordinate securely, directly from your dev machine.**



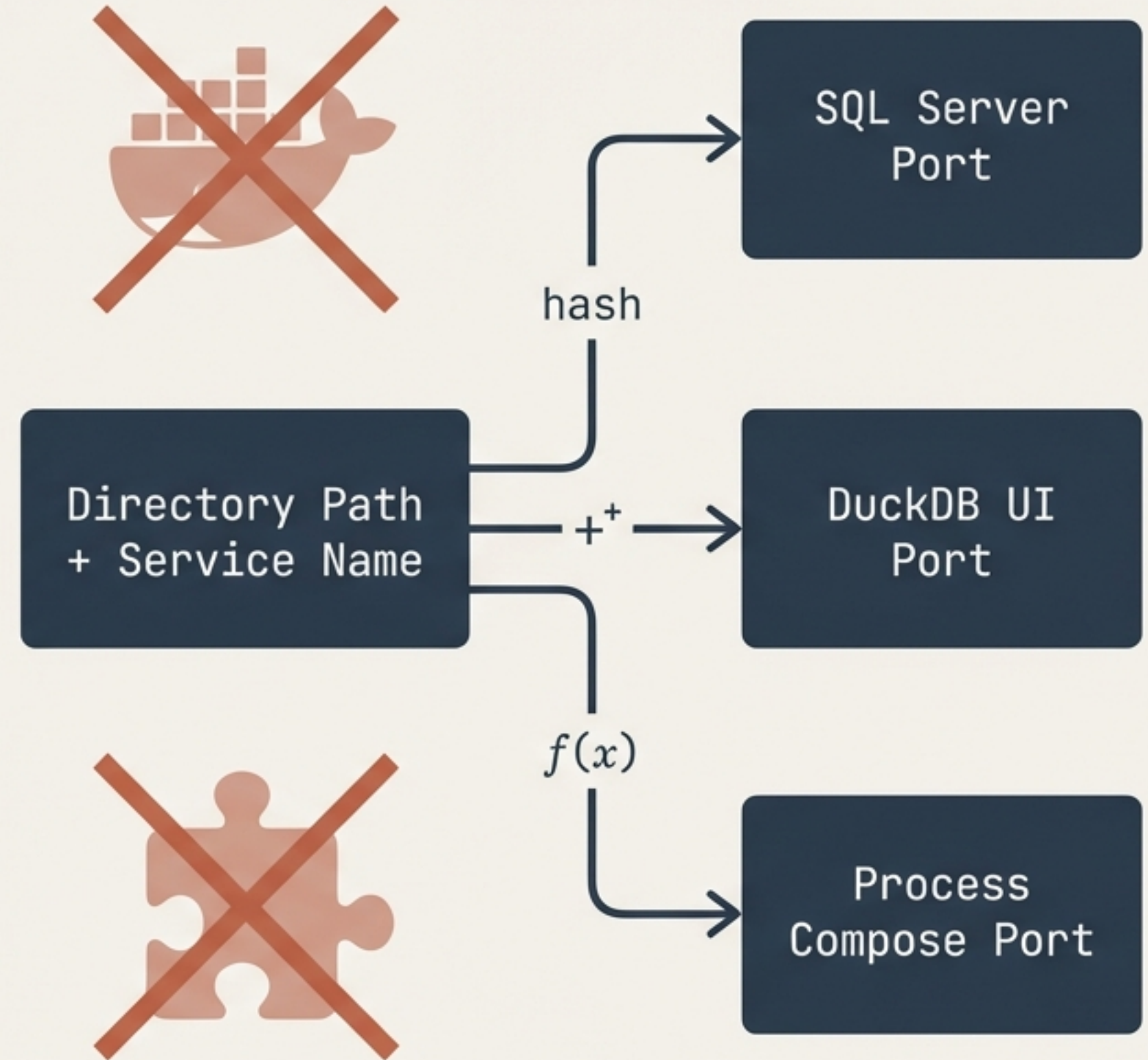
Unified Delivery Beyond Kubernetes

Helm hits a wall outside the cluster. BigConfig Packages treat infrastructure and software as a single unit. A single Clojure function effortlessly orchestrates Terraform and Ansible into one seamless deployment command.



Unifying Apps & Infrastructure

Abandon the YAML/EDN context switch. BigConfig System treats lifecycles as programmable workflows. It manages dynamic database initializations, polyglot credentials, and multi-port mappings entirely within a running Clojure session.

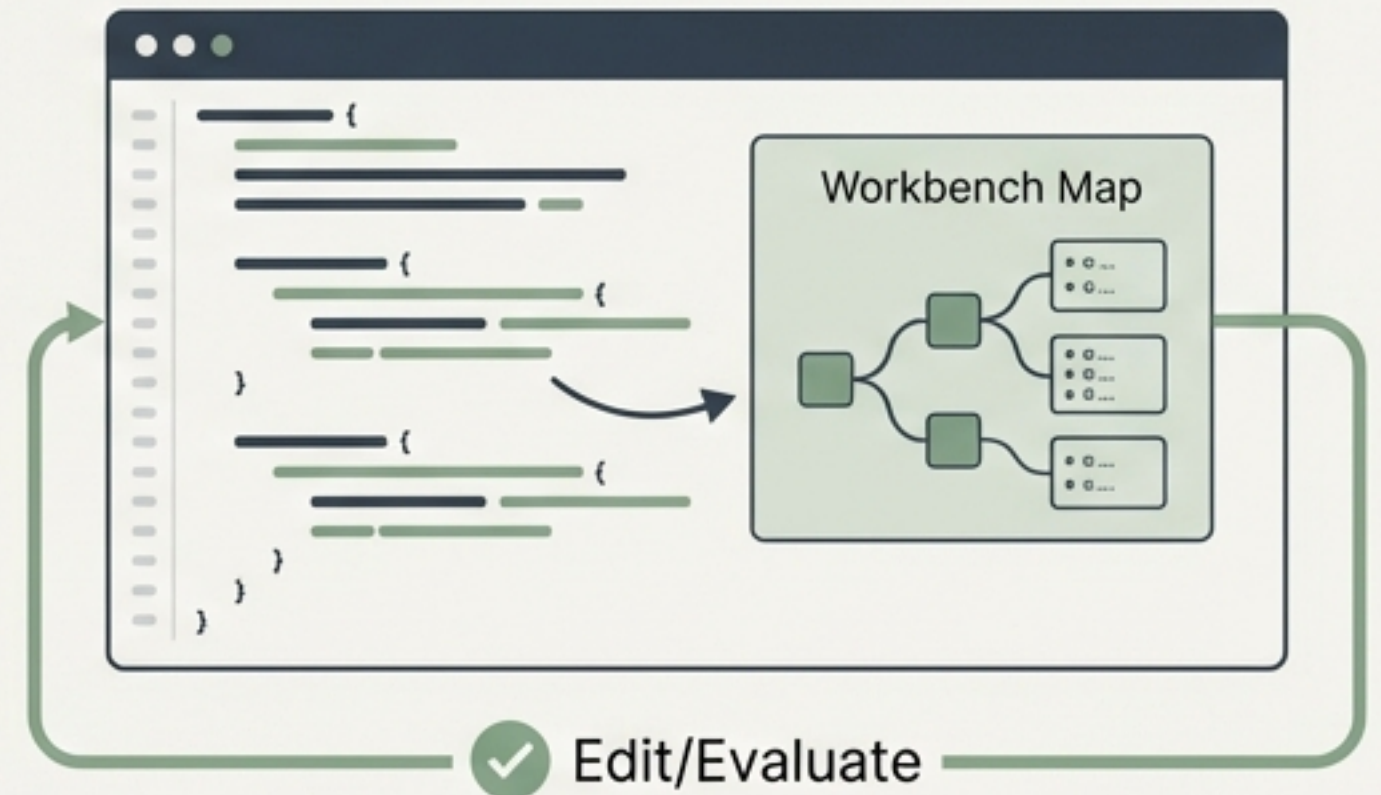


REPL-Driven Operations

Stop juggling twenty terminal tabs. BigConfig provides a persistent JVM workspace. Orchestrate tools, log gRPC traffic, and inspect state instantly via a centralized workbench map—no traditional debuggers required.



Chaotic Workspace
Charcoal Inter



Unified Workspace
Charcoal Inter

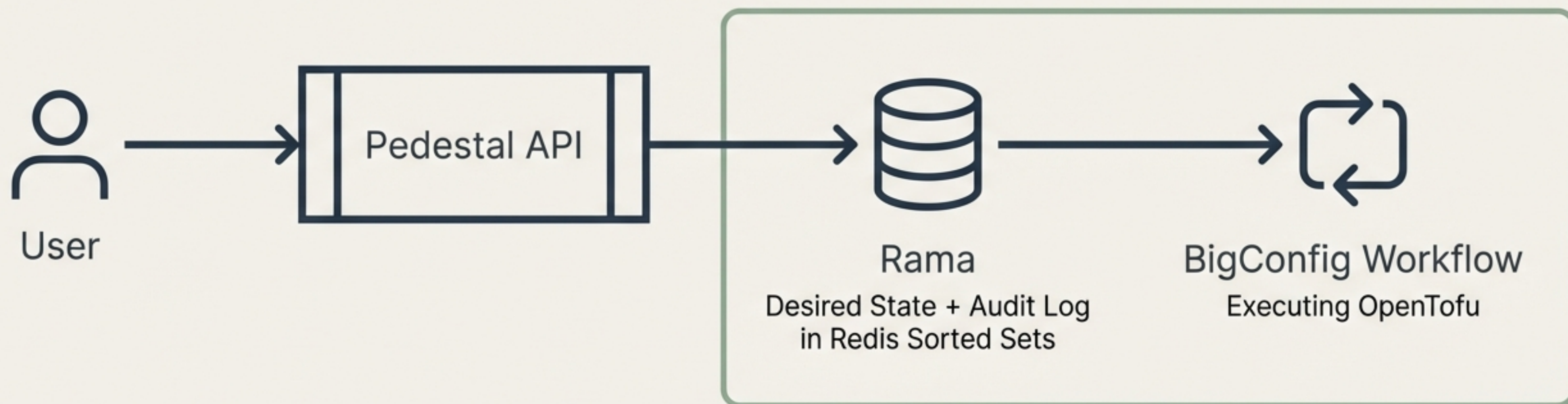
When GitOps Stops Scaling

One-time execution tools eventually fail to scale without continuous reconciliation. Relying solely on Git for state breaks down; modern platforms require a dedicated API to manage dynamic resource identifiers and enforce a continuous control loop.



Building a Control Plane in 200 Lines

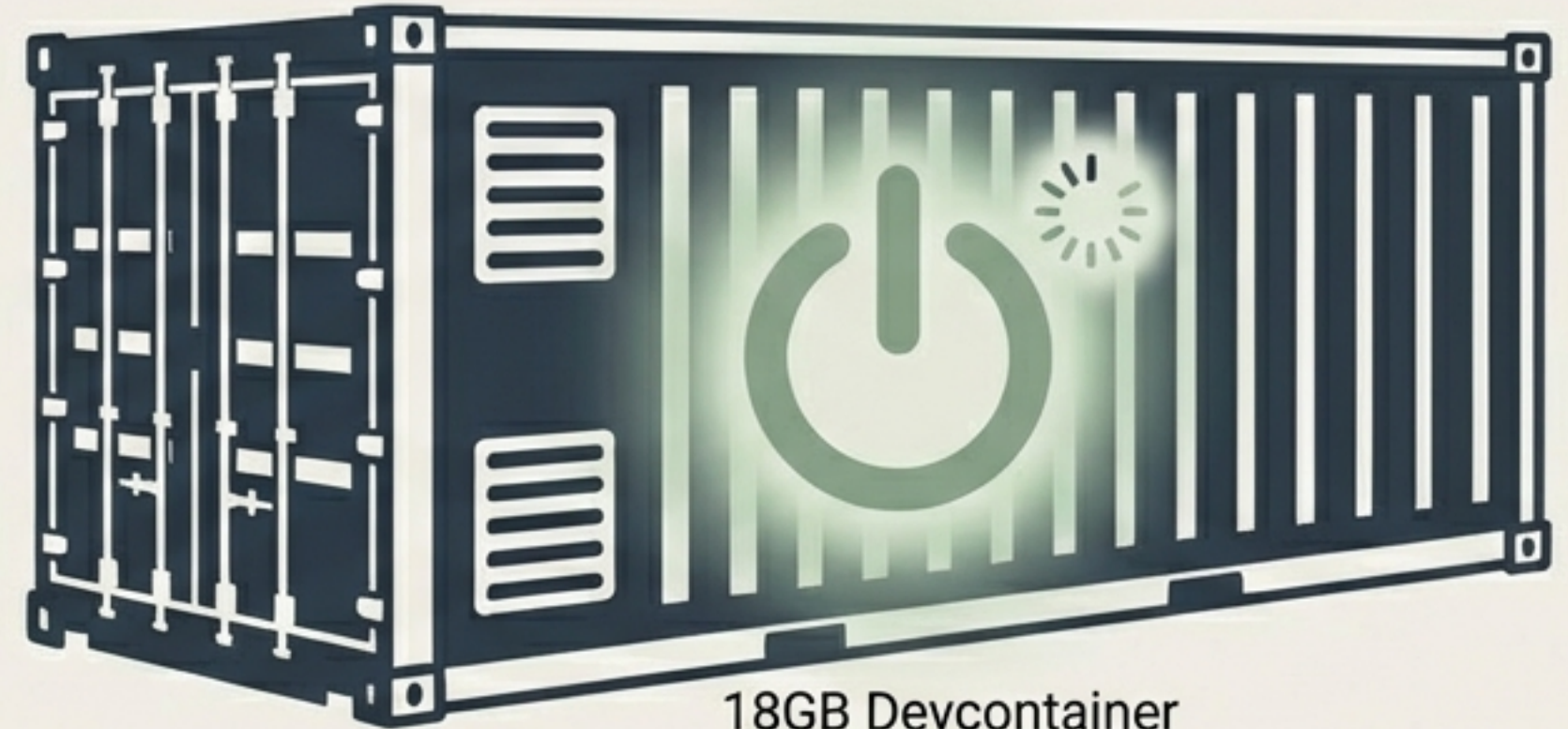
Upgrade from GitOps without rewriting your Terraform. By combining Pedestal for the API, Rama/Redis for immutable event-sourced state, and BigConfig for the execution loop, you can build a bespoke control plane effortlessly.



Local-First CI/CD Execution

Kill the slow commit-and-wait debugging cycle. By extracting CI steps to run inside standardized Docker devcontainers, BigConfig allows you to debug pipelines locally before ever pushing to GitHub Actions.

```
jet -i yaml -o json | jq -r  
  .jobs.test.steps[3].run | bash -s
```

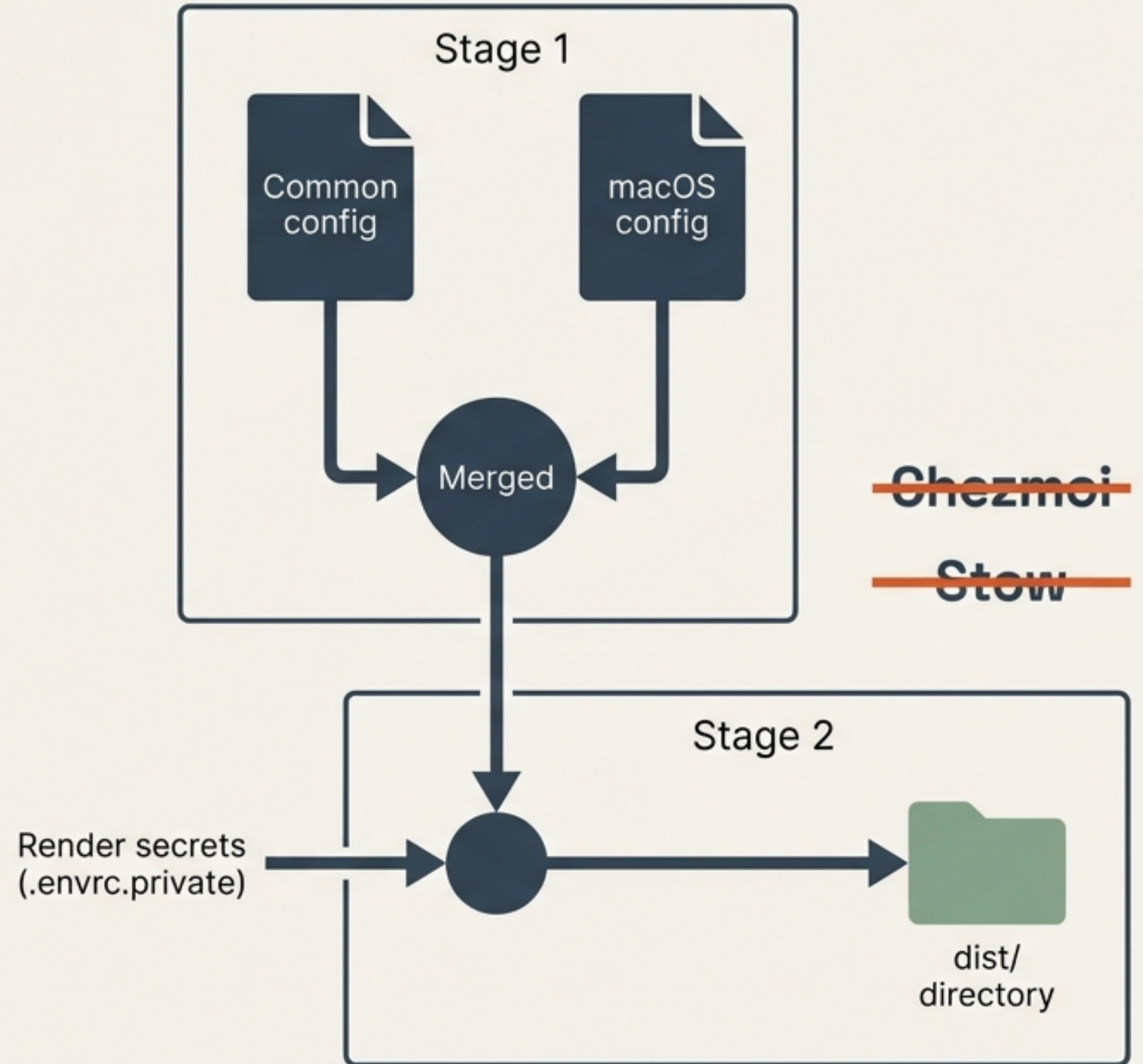


18GB Devcontainer
booting locally

Managing Dotfiles with Code

The same engine that deploys cloud infrastructure manages local development.

BigConfig replaces Chezmoi and Stow with a two-stage rendering pipeline, explicitly separating logic, platform-specific configs, and secrets.



Stop Curating. Start Programming.

Less YAML. Fewer fragmented CLIs. Faster inner loops.
Reclaim your operations with code-first infrastructure.

```
$ clojure -Tbig-config  
Available templates:  
  multi  
  terraform  
  ansible  
  dotfiles
```

Get started with built-in templates.
clojure -Tbig-config | github.com/amiorin/big-config